Estratégia Concursos

Aula 13

DataPrev (Analista de TI - Desenvolvimento de Software) Passo Estratégico de Conhecimentos Específicos - 2024 (Pós-Edital)

Autor:

09 de Setembro de 2024

11225354498 - Maria Vitória Gonçalves Vieira

Java Standard Edition

Sumário

Conteúdo 2
Glossário de termos 3
Roteiro de revisão 5
- Introdução e Histórico 5
- Arquitetura 7
- Tipos de Dados 11
- Operadores 12
- Palavras-chave 14
- Modificadores de Acessibilidade 15
- Estruturas de Repetição 16
- Estruturas de Seleção 17
- Programação OO: Classes e Objetos 19
- Interfaces 25
- Arrays 26
- Strings 30
- Exceções 31

Aposta estratégica 33
Questões Estratégicas 33
Questionário de revisão e aperfeiçoamento 42
- Perguntas 42
- Perguntas e Respostas 43

Lista de Questões Estratégicas 45

Conteúdo

Java. Introdução e histórico. Características. Arquitetura. Tipos e operadores. Palavras-chave. Modificadores de acesso. Estruturas de seleção e repetição. Classes e Objetos. Interfaces. Arrays. Strings. Exceções.

11225354498 - Maria Vitória Gonçalves Vieira

Análise Estatística

Inicialmente, convém destacar o percentual de incidência do assunto, dentro da disciplina Programação em concursos/cargos similares. Quanto maior o percentual de cobrança de um dado assunto, maior sua importância.

Obs.: *um mesmo assunto pode ser classificado em mais de um tópico devido à multidisciplinaridade de conteúdo*.

Assunto	Relevância na disciplina em concursos similares
Linguagens de programação	**35.7 %**
Python	**14.5 %**
JavaScript	**12.7 %**
Java	**3.6 %**
PHP	**2.5%**
VB Script	**1.8 %**
C Sharp	**1.8 %**
R	**1.8 %**
Web	**5.5 %**
Linguagens de marcação	**5.5 %**
XML (Extensible Markup Language)	**3.6 %**
HTML (HyperText Markup Language)	**1.8 %**
CSS (Cascading Style Sheets)	**3.6 %**
Conceitos básicos de programação	**1.8 %**
Frameworks Java	**1.8 %**
Hibernate	**1.8 %**

11225354498 - Maria Vitória Gonçalves Vieira

GLOSSÁRIO DE TERMOS

Faremos uma lista de termos que são relevantes ao entendimento do assunto desta aula. Caso tenha alguma dúvida durante a leitura, esta seção pode lhe ajudar a esclarecer.

Java: Uma linguagem de programação orientada a objetos, de alto nível e de propósito geral, projetada para ser simples e segura. É usada principalmente para desenvolvimento de software do lado do servidor, desenvolvimento de software móvel (Android) e software embarcado.

JVM (Máquina Virtual Java): Uma máquina virtual que fornece um ambiente de tempo de execução no qual o bytecode Java pode ser executado. A JVM é uma parte essencial da tecnologia Java e fornece independência de plataforma.

Bytecode: O código intermediário para o qual o código fonte Java é compilado pelo compilador Java. O bytecode pode ser interpretado (ou executado diretamente) por JVMs em qualquer plataforma que possua uma JVM instalada.

Garbage Collector: Uma parte da JVM que automaticamente recupera a memória que não é mais necessária em tempo de execução, liberando o programador da necessidade de desalocar manualmente a memória.

Tipo Primitivo: Em Java, um tipo de dado predefinido, como int, char, float, boolean etc., que armazena um valor único. Os tipos primitivos não são objetos e, portanto, não têm métodos.

Tipo de Referência: Em Java, são os tipos de dados que são criados pelo usuário (como classes, interfaces) e os tipos predefinidos (como String, objetos, arrays etc.). Os tipos de referência têm valor null por padrão.

abstract: Uma palavra-chave em Java que é usada para declarar uma classe ou método abstrato. Uma classe abstrata não pode ser instanciada e um método abstrato não tem corpo e deve ser implementado em uma subclasse.

catch: Uma palavra-chave usada para definir um bloco que é usado para lidar com exceções em Java. Ele deve ser usado após o bloco try.

extends: Uma palavra-chave usada para declarar uma nova classe que herda de uma classe pai em Java.

11225354498 - Maria Vitória Gonçalves Vieira

final: Uma palavra-chave usada para declarar uma constante. Pode ser aplicada a variáveis, métodos e classes. Uma variável final não pode ser modificada, um método final não pode ser sobrescrito e uma classe final não pode ser estendida.

finally: Uma palavra-chave usada para criar um bloco que será executado após um bloco try ou catch, independentemente de uma exceção ser ou não lançada ou capturada.

implements: Uma palavra-chave usada para implementar uma interface em uma classe em Java.

instanceof: Uma palavra-chave que é usada para testar se um objeto é uma instância de uma classe, uma subclasse ou uma interface.

new: Uma palavra-chave usada para criar uma nova instância de uma classe ou um novo array em Java.

static: Uma palavra-chave usada para indicar que um membro de uma classe pertence à classe em vez de uma instância da classe.

try: Uma palavra-chave usada para definir um bloco que vai tentar executar um código que pode lançar uma exceção.

void: Uma palavra-chave usada para indicar que um método não retorna valor.

public: Um modificador de acesso que permite que uma classe, método, ou campo seja acessível de qualquer outra classe.

private: Um modificador de acesso que permite que uma classe, método, ou campo seja acessível apenas dentro da sua classe de definição.

protected: Um modificador de acesso que permite que uma classe, método, ou campo seja acessível dentro do pacote onde está definido, e também por subclasses de sua classe em qualquer pacote.

Visibilidade default: Se nenhum modificador de acesso é especificado, o acesso é default, o que permite que uma classe, método, ou campo seja acessível dentro do pacote onde está definido.

this: Uma palavra-chave que se refere à instância atual de uma classe. Pode ser usada para acessar campos de classe, invocar métodos de classe e como uma referência ao objeto atual.



11225354498 - Maria Vitória Gonçalves Vieira

Roteiro de revisão

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Introdução e Histórico

Java foi desenvolvido inicialmente em 1991 por uma equipe liderada por James Gosling na Sun Microsystems, que mais tarde foi adquirida pela Oracle Corporation. A linguagem foi originalmente chamada de "Oak", mas foi renomeada para "Java" em 1995. Seu nome é uma referência à variedade de café Java, um indicativo da alta quantidade de café consumida pelos desenvolvedores durante o projeto.

Java foi criada com o objetivo de permitir que desenvolvedores escrevessem um programa uma vez e o executassem em qualquer dispositivo. Esta filosofia é resumida no slogan "write once, run anywhere" (WORA), ou seja, escreva uma vez, execute em qualquer lugar.

A primeira versão pública, Java 1.0, foi lançada em 1996 com o objetivo de fornecer uma linguagem simples, orientada a objetos, distribuída, interpretada, robusta, segura, arquitetonicamente neutra, portátil, de alto desempenho, multithread e dinâmica. Desde então, Java passou por várias adições ao longo dos anos, com novas classes, pacotes, métodos e recursos sendo constantemente adicionados à linguagem.

Java 2 foi lançado em 1998 e introduziu a distinção entre as edições Standard Edition (SE), Enterprise Edition (EE) e Micro Edition (ME). Esta versão também introduziu a coleção de frameworks, que são um recurso central do Java hoje.

Java 5 foi lançado em 2004 e introduziu várias melhorias significativas, incluindo a introdução do recurso de autoboxing/unboxing, enumerações, varargs e a introdução do for-each loop. Além disso, foram adicionados os recursos de concorrência, o que tornou a linguagem mais amigável para aplicações multithreaded.

11225354498 - Maria Vitória Gonçalves Vieira

Java 8, lançado em 2014, foi outra grande atualização. Ela trouxe as expressões lambda e os streams, que foram uma mudança significativa para a linguagem, introduzindo a programação funcional no Java.

Em 2017, a Oracle anunciou que adotaria um novo ciclo de lançamento de seis meses para Java, e desde então, temos visto atualizações regulares a cada seis meses.

A Oracle, depois de adquirir a Sun Microsystems em 2010, manteve a continuidade do desenvolvimento de Java e deu continuidade à política de atualizações constantes e regulares, garantindo que Java continue sendo uma linguagem relevante e amplamente utilizada.

Características

Java é conhecido por algumas características importantes. Vejamos cada uma delas:

- Portabilidade: Java foi projetada com o lema "escreva uma vez, execute em qualquer lugar" (write once, run anywhere - WORA). Isso se deve ao fato de que o código Java é compilado em bytecode, que pode ser executado em qualquer máquina que possua uma Java Virtual Machine (JVM), independentemente do sistema operacional subjacente. Isso torna Java altamente portável.
- Orientação a Objetos: Java é uma linguagem puramente orientada a objetos. Isso significa que tudo em Java é um objeto. A programação orientada a objetos (OOP) permite que os desenvolvedores criem aplicativos modularizados e reutilizáveis, além de modelar objetos do mundo real dentro de um contexto de software.
- Segurança: Java foi projetada com a segurança em mente. A linguagem fornece vários recursos de segurança, como ausência de ponteiros explícitos, verificação de tipo em tempo de compilação, sandbox de segurança para executar código não confiável, além de várias ferramentas e APIs de segurança.
- Desempenho: Java tem um desempenho relativamente bom graças à JVM, que é altamente otimizada. A introdução do compilador Just-In-Time (JIT) na JVM melhorou ainda mais o desempenho, compilando o bytecode em tempo de execução para código de máquina nativo.
- Robustez: Java é considerada robusta devido ao seu forte sistema de verificação de tipo, gerenciamento automático de memória (Garbage Collection), e o tratamento de exceções, que ajuda a prevenir erros em tempo de execução e a aumentar a confiabilidade do software.
- Simplicidade: Embora Java seja uma linguagem poderosa, ela foi projetada para ser fácil de aprender e usar, especialmente para desenvolvedores com experiência em C++. Ela elimina vários recursos de linguagens anteriores, como ponteiros e

11225354498 - Maria Vitória Gonçalves Vieira

operadores de sobrecarga, que podem complicar a programação. Além disso, a sintaxe de Java é limpa e fácil de seguir, o que contribui para a sua simplicidade.

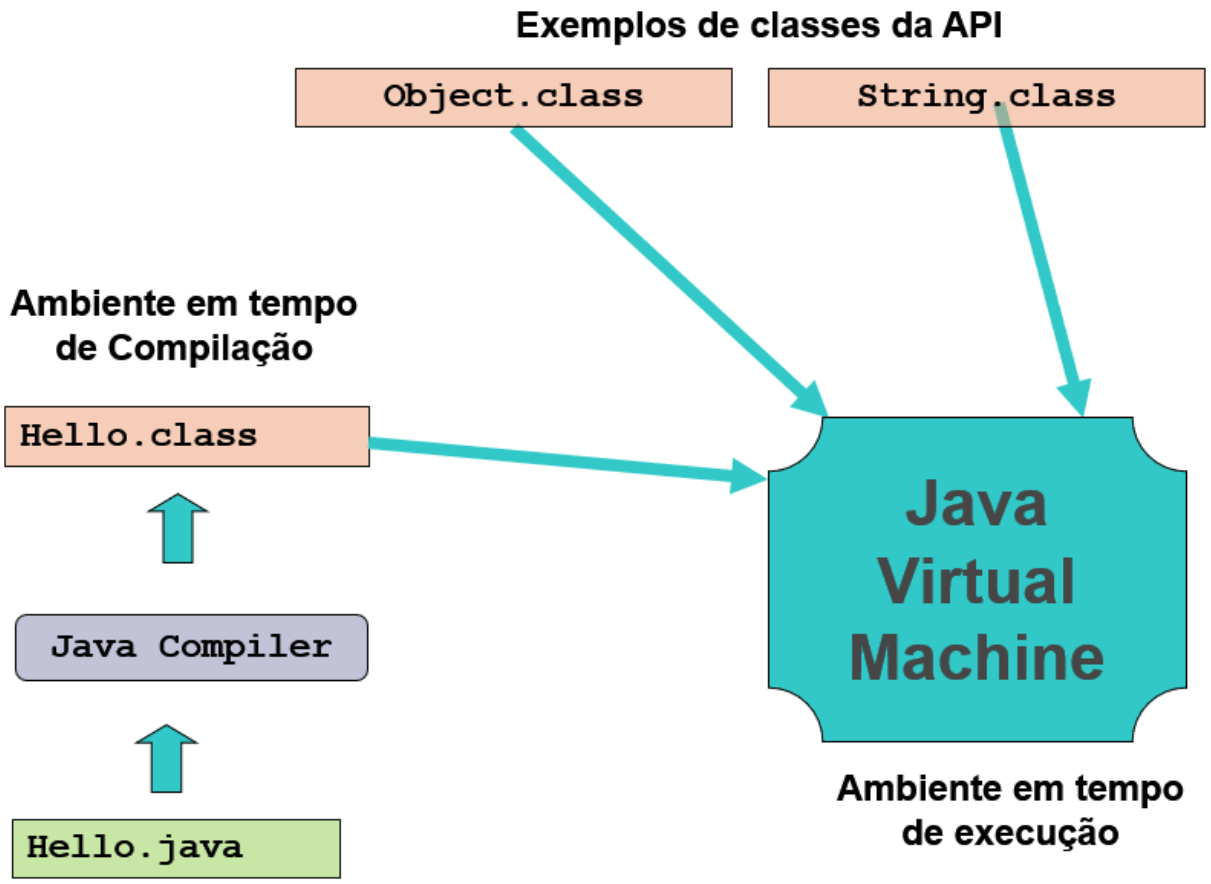
Arquitetura

Para entendermos a arquitetura de execução Java, é importante estudarmos o modelo de execução da JVM (Máquina Virtual Java), o Bytecode e seu papel na portabilidade e o papel do Garbage Collector no ecossistema Java.

JVM - Java Virtual Machine

A JVM é uma máquina abstrata, ou seja, um modelo teórico que fornece um ambiente de execução no qual o código Java (bytecode) pode ser executado. A JVM age como uma ponte entre o código e o hardware subjacente, permitindo que Java seja executada independentemente do sistema operacional ou do tipo de hardware em que está sendo executada.

11225354498 - Maria Vitória Gonçalves Vieira



Vejamos mais de perto o funcionamento da JVM:

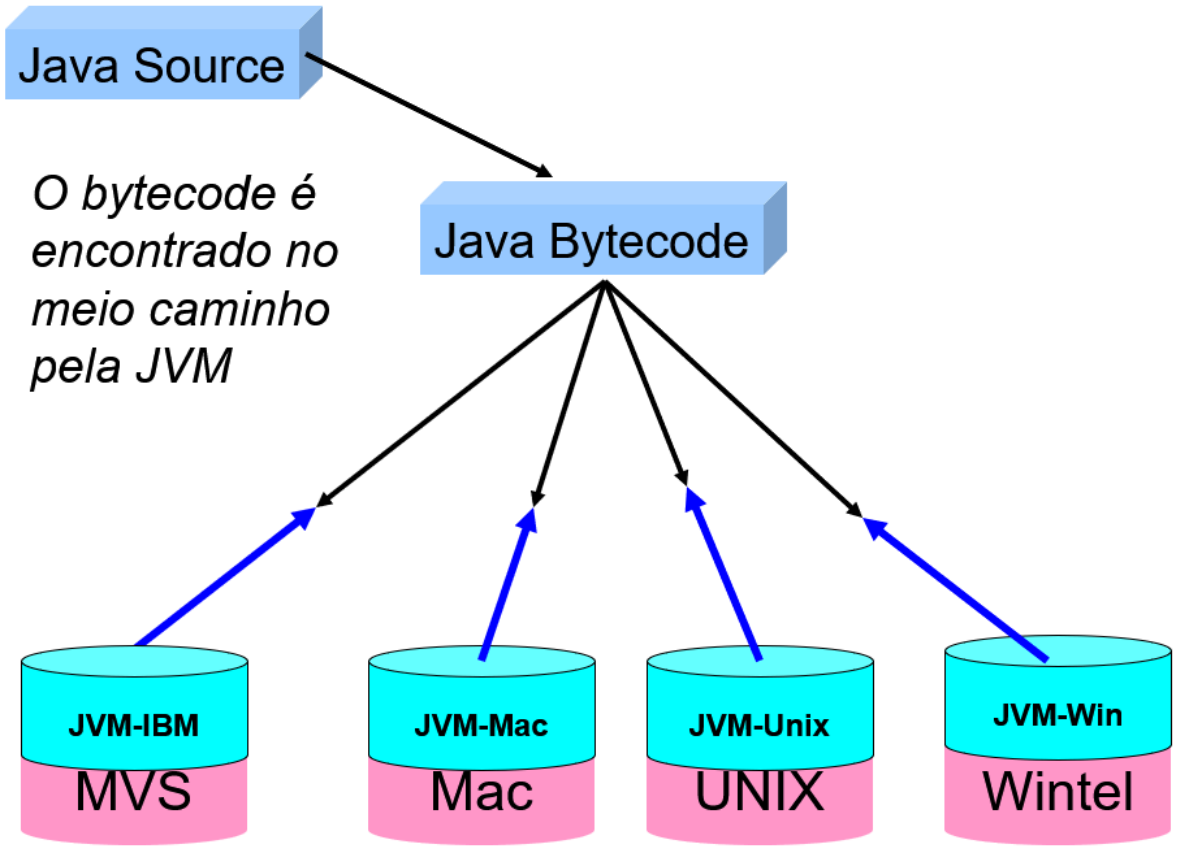
- Carga de Classe (Class Loading): A JVM carrega as classes necessárias no momento da execução. O carregador de classes localiza e lê os arquivos .class (que contêm o bytecode Java) e os carrega na memória.
- Verificação de Bytecode (Bytecode Verification): Uma vez carregadas as classes, a JVM realiza uma verificação completa do bytecode para garantir que não contém nada prejudicial ou inapropriado (como a violação das restrições de acesso de objeto) e que é formatado corretamente. Esse processo é parte do compromisso de Java com a segurança.
- Execução: O bytecode é então executado pela JVM. Em muitos casos, o bytecode é compilado em tempo de execução em código de máquina nativo para o hardware específico no qual está sendo executado, para melhorar o desempenho. Isso é feito por uma parte da JVM chamada de compilador Just-In-Time (JIT).
- Gerenciamento de Memória (Garbage Collection): A JVM também gerencia a memória automaticamente, o que significa que os desenvolvedores não precisam se preocupar com a alocação e a liberação de memória. Isso é feito por meio de um

processo chamado "Garbage Collection" (GC), que automaticamente localiza e libera memória que não está mais em uso pelo programa.

- Encerramento: Finalmente, quando o programa termina (ou seja, quando todas as threads terminaram ou quando o método exit é chamado), a JVM limpa todos os recursos e se encerra de maneira ordenada.

Bytecode

Bytecode é uma forma intermediária de instruções que é gerada quando o código-fonte Java é compilado pelo compilador Java (javac). Este bytecode é uma representação otimizada do código-fonte Java, projetada para ser interpretada rapidamente. Ele é armazenado em arquivos com a extensão .class.



O principal papel do bytecode é contribuir para a portabilidade de Java. Como mencionado anteriormente, o lema de Java é "escreva uma vez, execute em qualquer lugar" (write once, run anywhere - WORA). Isso é possível graças ao bytecode.

Quando você compila o código-fonte Java em bytecode, ele pode ser executado em qualquer sistema que tenha a Java Virtual Machine (JVM) instalada,

independentemente da arquitetura do sistema operacional ou do hardware subjacente. A JVM interpreta o bytecode e o executa na plataforma host. Em muitos casos, a JVM também usa um compilador Just-In-Time (JIT) para traduzir o bytecode em código de máquina nativo no momento da execução, para aumentar a eficiência.

Garbage Collector

O Garbage Collector, ou Coletor de Lixo, é uma parte essencial da Java Virtual Machine (JVM) que gerencia automaticamente a memória em um programa Java. Ele libera o desenvolvedor da tarefa de gerenciar manualmente o ciclo de vida do objeto, uma fonte comum de erros e problemas de desempenho em muitas linguagens de programação.

Na linguagem Java, quando um objeto é criado, a memória é alocada para ele na heap, que é uma área de memória reservada para a alocação dinâmica. Conforme o programa continua a executar e novos objetos são criados, mais e mais memória é usada. No entanto, nem todos os objetos permanecem úteis durante todo o tempo de vida do programa. Quando objetos não são mais necessários, eles devem ser removidos para liberar memória.

O Garbage Collector tem a função de encontrar esses objetos que não são mais acessíveis ou necessários e liberar a memória que eles estão ocupando. Ele faz isso por meio de um processo chamado garbage collection.

O processo de garbage collection geralmente envolve as seguintes etapas:

1. Marcação (Marking): Nesta etapa, o Garbage Collector identifica quais peças de memória estão em uso e quais não estão.
2. Exclusão Normal (Normal Deletion): A memória não usada é liberada. É importante notar que o espaço de memória coletado não é desalocado, mas sim adicionado ao 'free pool', pronto para ser reutilizado.
3. Exclusão com Compactação (Deletion with Compacting): Para resolver o problema de fragmentação de memória, o Garbage Collector pode optar por mover os objetos referenciados para um lado da heap, de modo que todo o espaço livre esteja contíguo.

O momento exato de quando o garbage collection ocorrerá é decidido pela JVM. Na prática, isso geralmente ocorre quando a heap está quase cheia, ou em momentos de ociosidade do sistema.

11225354498 - Maria Vitória Gonçalves Vieira

Tipos de Dados

Em Java, existem dois tipos principais de dados: tipos primitivos e tipos de referência.

1. Tipos Primitivos: Esses são os tipos de dados básicos que incluem byte, short, int, long, float, double, boolean e char. Cada tipo primitivo tem um tamanho e um valor padrão. Por exemplo, o tamanho de int é 32 bits e seu valor padrão é 0, enquanto o tamanho de boolean é virtualmente indefinido pela especificação Java e seu valor padrão é false. Os tipos primitivos sempre têm um valor, não podem ser null e são alocados na stack.
2. Tipos de Referência: Ao contrário dos tipos primitivos, os tipos de referência são criados pelo programador (como classes e interfaces) ou fornecidos pela linguagem Java (como String e Array). Eles são chamados de tipos de referência porque eles contêm referências (ou endereços) para os locais na memória onde os dados reais estão armazenados. Os tipos de referência são alocados na heap e podem ser null.

Veja alguns exemplos importantes dos tipos de referência:

- Classes: Em Java, todas as classes são tipos de referência. Elas podem incluir campos de dados primitivos e/ou referências para outros objetos.
- Interfaces: As interfaces também são tipos de referência em Java. Uma interface é como um modelo que pode ser usado para projetar as classes.
- Arrays: Em Java, os arrays são objetos, não tipos primitivos. Eles contêm um número fixo de valores do mesmo tipo. Um array pode conter tipos primitivos ou referências a objetos.
- String: Em Java, String é um tipo de referência, embora tenha algumas características únicas que podem fazer com que pareça um tipo primitivo. Por exemplo, os valores de string podem ser atribuídos usando aspas duplas, que normalmente são usadas para valores primitivos em muitas linguagens.

A tabela abaixo lista os tipos primitivos de Java e suas respectivas características:

Tipo	Descrição
boolean	**Pode assumir o valor true ou o valor false. Representa números binários.**
byte	**Inteiro de 8 bits em notação de complemento de dois. Pode assumir valores entre -2^7=-128 e 2^7-1=127.**

11225354498 - Maria Vitória Gonçalves Vieira

short	**Inteiro de 16 bits em notação de complemento de dois. Os valores possíveis cobrem a faixa de -2^{-15}=-32.768 a 2^{15}-1=32.767**
char	**Caractere em notação Unicode de 16 bits. Serve para o armazenamento de dados alfanuméricos. Também pode ser usado como um dado inteiro com valores na faixa entre 0 e 65535.**
int	**Inteiro de 32 bits em notação de complemento de dois. Pode assumir valores entre -2^{31}=2.147.483.648 e 2^{31}-1=2.147.483.647**
long	**Inteiro de 64 bits em notação de complemento de dois. Pode assumir valores entre -2^{63} e 2^{63}-1.**
float	**Representa números em notação de ponto flutuante normalizada em precisão simples de 32 bits**
double	**Representa números em notação de ponto flutuante normalizada em precisão dupla de 64 bits**

Operadores

Os operadores em Java são usados para realizar operações em variáveis e valores. A linguagem fornece vários operadores, que podem ser classificados em algumas categorias: aritméticos, relacionais, lógicos e de atribuição.

1. Operadores Aritméticos: Esses operadores são usados para realizar operações matemáticas básicas. Eles incluem:
    - + (adição)
    - - (subtração)
    - * (multiplicação)
    - / (divisão)
    - % (módulo, ou operador de resto)
2. Operadores Relacionais: Os operadores relacionais são usados para comparar dois valores e retornar um valor booleano (verdadeiro ou falso) com base no resultado da comparação. Eles incluem:
    - == (igual a)
    - != (não igual a)

11225354498 - Maria Vitória Gonçalves Vieira

- > (maior que)
- < (menor que)
- >= (maior ou igual a)
- <= (menor ou igual a)

3. Operadores Lógicos: Os operadores lógicos são usados para realizar operações lógicas (AND, OR e NOT) em valores booleanos. Eles incluem:
   - && (AND lógico)
   - || (OR lógico)
   - ! (NOT lógico)
4. Operadores de Atribuição: Os operadores de atribuição são usados para atribuir um valor a uma variável. O operador de atribuição mais simples é =, mas existem vários operadores de atribuição compostos que realizam uma operação aritmética ou bitwise e, em seguida, atribuem o resultado à variável. Eles incluem:
   - = (atribuição)
   - += (adição e atribuição)
   - -= (subtração e atribuição)
   - *= (multiplicação e atribuição)
   - /= (divisão e atribuição)
   - %= (módulo e atribuição)

Veja está uma tabela mostrando a precedência dos operadores em Java. Os operadores são listados de cima para baixo em ordem decrescente de precedência.

Precedência	Categoria do Operador	Operadores
1	operadores pós-fixados	expr++, expr--
2	operadores unários prefixados	++expr, --expr, +expr, -expr, ~, !
3	operadores de criação e cast	new, (tipo)
4	operadores multiplicativos	*, /, %
5	operadores aditivos	+, -
6	operadores de shift	<<, >>, >>>
7	operadores relacionais	<, <=, >, >=, instanceof
8	operadores de igualdade	==, !=
9	operador bitwise AND	&
10	operador bitwise XOR	^
11	operador bitwise OR	|
12	operador lógico AND	&&

11225354498 - Maria Vitória Gonçalves Vieira

13	operador lógico OR	||
14	operador ternário	?:
15	operadores de atribuição	=, +=, -=, *=, /=, %=, &=, ^=, |=, <<=, >>=, >>>=

Os operadores na mesma linha têm a mesma precedência. Quando operadores com a mesma precedência aparecem em uma expressão, a associação (a ordem na qual as operações são realizadas) depende da associatividade dos operadores. Na maioria dos casos, os operadores em Java são associativos à esquerda, o que significa que as operações são realizadas da esquerda para a direita. No entanto, o operador de atribuição e o operador ternário são associativos à direita.

Embora a precedência e a associação dos operadores sejam importantes, é sempre uma boa prática usar parênteses para tornar o código mais legível e evitar possíveis confusões.

Palavras-chave

Palavras-chave em Java são palavras reservadas que têm um significado específico para o compilador e não podem ser usadas para qualquer outra coisa, como nomes de variáveis ou de métodos. Cada palavra-chave tem uma função específica no idioma.

Aqui está uma tabela com as principais palavras-chave reservadas em Java que são cobradas em concurso:

Palavra-chave	Descrição
abstract	Usada para declarar uma classe abstrata ou um método abstrato, que não possui implementação e deve ser implementado por qualquer classe que não seja abstrata que a herde.
catch	Usada em conjunto com try para capturar e tratar exceções.
extends	Usada na declaração de classe para herdar de uma superclasse.
final	Usada para indicar que um valor é constante e não pode ser alterado, ou que uma classe não pode ser herdada, ou que um método não pode ser sobrescrito.

11225354498 - Maria Vitória Gonçalves Vieira

finally	**Usada após um bloco try ou catch para especificar um bloco de código que será executado independentemente de uma exceção ser ou não lançada ou capturada.**
implements	**Usada na declaração de classe para herdar de uma ou mais interfaces.**
instanceof	**Usada para testar se um objeto é uma instância de uma classe específica, de uma subclasse ou de uma interface específica.**
new	**Usada para criar uma nova instância de um objeto ou array.**
static	**Usada para declarar uma variável de classe (em vez de uma variável de instância) ou um método de classe.**
try	**Usada para envolver um bloco de código em que uma exceção pode ocorrer. Em seguida, o bloco de código é seguido por um ou mais blocos catch ou um bloco finally.**
void	**Usada na declaração de método para indicar que o método não retorna nenhum valor.**

Modificadores de Acessibilidade

Modificadores de acessibilidade em Java são usados para definir a visibilidade dos membros de uma classe (campos, métodos, etc.). Eles permitem que você controle onde esses membros podem ser acessados de dentro do código do programa. Java possui quatro modificadores de acessibilidade: private, public, protected e default.

Veja uma tabela com uma breve descrição de cada um:

Modificador	Descrição
public	**O membro é acessível de qualquer lugar. Isto inclui outras classes e pacotes.**
private	**O membro só pode ser acessado dentro da própria classe. Outras classes, subclasses e pacotes não podem acessá-lo.**
protected	**O membro pode ser acessado na própria classe, em suas subclasses e no mesmo pacote de classes. Porém, não é acessível em um pacote diferente daquele onde está localizado, a menos que seja por meio de uma subclasse.**
(nenhum, ou default)	**Se nenhum modificador de acesso é especificado, então o acesso é permitido dentro da própria classe e dentro do mesmo pacote de classes. No entanto, o membro não é acessível em uma classe de um pacote diferente ou em subclasses em pacotes diferentes.**



11225354498 - Maria Vitória Gonçalves Vieira

Estruturas de Repetição

Estruturas de repetição, ou loops, são usadas para repetir um bloco de código várias vezes. Java suporta três tipos principais de estruturas de repetição: for, while e do-while.

For: O loop for é comumente usado quando sabemos o número exato de vezes que um bloco de código deve ser executado. A sintaxe geral é for(inicialização; condição; passo).

```
for(int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

Neste exemplo, a variável i é inicializada com 0, a condição para a continuação do loop é i < 5 e o passo é i++, que incrementa o valor de i em 1 a cada iteração. O bloco de código dentro do loop será executado enquanto a condição for verdadeira.

While: O loop while é usado quando queremos que um bloco de código seja repetido enquanto uma condição for verdadeira. A condição é verificada antes de entrar no loop.

```
int i = 0;
while(i < 5) {
    System.out.println(i);
    i++;
}
```

Neste exemplo, o loop continuará enquanto i < 5 for verdadeiro. Observe que o incremento de i está dentro do bloco de código do loop while.

Do-While: O loop do-while é muito semelhante ao loop while, mas com uma diferença crucial: a condição é verificada após a execução do bloco de código. Isso significa que o bloco de código será executado pelo menos uma vez, independentemente da condição.

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while(i < 5);
```

Neste exemplo, o bloco de código será executado primeiro, e só então a condição i < 5 será verificada.

Estruturas de Seleção

Estruturas de seleção, ou comandos condicionais, são usadas para executar diferentes blocos de código com base em condições diferentes. Java suporta duas estruturas principais de seleção: if-else e switch.

If-Else: A estrutura de seleção if-else é usada para escolher entre dois blocos de código com base no resultado de uma condição. A sintaxe geral é if(condição) {bloco de código} else {bloco de código}.

```
int x = 10;
if(x > 5) {
    System.out.println("x é maior que 5");
} else {
    System.out.println("x não é maior que 5");
}
```

Neste exemplo, a condição é x > 5. Se a condição for verdadeira, o bloco de código dentro do if será executado. Se a condição for falsa, o bloco de código dentro do else será executado.

Você pode também usar múltiplos else if para testar múltiplas condições, como no exemplo abaixo:

11225354498 - Maria Vitória Gonçalves Vieira

```java
int x = 10;
if(x > 10) {
    System.out.println("x é maior que 10");
} else if(x == 10) {
    System.out.println("x é igual a 10");
} else {
    System.out.println("x é menor que 10");
}
```

Switch: A estrutura de seleção switch é usada para escolher entre vários blocos de código com base no valor de uma variável. A sintaxe geral é switch(variável) {case valor1: bloco de código; break; case valor2: bloco de código; break; ... default: bloco de código;}.

```java
int x = 1;
switch(x) {
    case 1:
        System.out.println("x é 1");
        break;
    case 2:
        System.out.println("x é 2");
        break;
    default:
        System.out.println("x não é 1 nem 2");
}
```

Neste exemplo, o valor de x é comparado com cada um dos casos. Se x for igual a um dos casos, o bloco de código correspondente será executado. Se x não for igual a nenhum dos casos, o bloco de código dentro do default será executado. O comando break é usado para sair do switch depois que um caso correspondente foi encontrado e executado.

11225354498 - Maria Vitória Gonçalves Vieira

Programação OO: Classes e Objetos

Em Java, uma **classe** é uma estrutura de dados que combina estado (atributos) e comportamento (métodos). É um plano ou um modelo a partir do qual os objetos são criados. A classe define o que o objeto será, mas não é o objeto em si. Por exemplo, podemos ter uma classe **Carro** que define atributos como **cor** e **marca**, e métodos como **acelerar()** e **frear()**.

```
public class Carro {
    // Atributos
    String cor;
    String marca;

    // Métodos
    void acelerar() {
        // código para acelerar
    }

    void frear() {
        // código para frear
    }
}
```

Um **objeto** é uma instância de uma classe. Quando você cria um objeto, está criando uma entidade que possui um estado e um comportamento definidos pela sua classe. Cada objeto tem seu próprio estado e acesso aos comportamentos definidos pela sua classe.

```
Carro meuCarro = new Carro();
meuCarro.cor = "vermelho";
meuCarro.marca = "Ferrari";
```

Neste exemplo, **meuCarro** é um objeto da classe **Carro**. Tem um estado (cor vermelha e marca Ferrari) e tem acesso aos métodos **acelerar()** e **frear()**.

11225354498 - Maria Vitória Gonçalves Vieira

Atributos: Os atributos são variáveis que guardam o estado do objeto. Eles são definidos na classe e cada objeto terá sua própria cópia dessas variáveis. No exemplo acima, cor e marca são atributos da classe Carro.

Métodos: Os métodos são funções definidas na classe que descrevem o comportamento dos objetos dessa classe. Eles podem modificar o estado do objeto ou realizar alguma computação com base no estado atual do objeto. No exemplo acima, acelerar() e frear() são métodos da classe Carro.

Instanciação de objetos

A instanciação de objetos é o processo de criação de uma instância de uma classe, ou seja, um objeto. Em Java, a palavra-chave new é usada para instanciar objetos de uma classe. Quando um objeto é instanciado, o construtor da classe é chamado e a memória necessária para o objeto é alocada.

Por exemplo, considere a classe Carro definida anteriormente. Podemos instanciar um objeto dessa classe da seguinte maneira:

```
Carro meuCarro = new Carro();
```

Neste exemplo, new Carro() chama o construtor da classe Carro para criar um novo objeto, e meuCarro é uma referência a esse objeto. Podemos agora usar meuCarro para acessar os atributos e métodos do objeto.

Palavra-chave 'this':

A palavra-chave this em Java é uma referência à instância atual de um objeto. É usada dentro dos métodos e construtores de uma classe para referir-se ao objeto atual.

Por exemplo, você pode usar this para referir-se aos atributos do objeto em um construtor:

11225354498 - Maria Vitória Gonçalves Vieira

```java
public class Carro {
    String cor;

    Carro(String cor) {
        this.cor = cor;  // 'this.cor' refere-se ao atributo 'cor' do objeto.
    }
}
```

Neste exemplo, this.cor é usado para diferenciar o atributo cor do objeto do parâmetro cor do construtor.

A palavra-chave this também pode ser usada para chamar um método da instância atual:

```java
public class Carro {
    void acelerar() {
        // código para acelerar
    }

    void iniciarCorrida() {
        this.acelerar();
    }
}
```

Neste exemplo, this.acelerar() chama o método acelerar() da instância atual do objeto Carro. Embora o uso de this seja opcional neste caso, pode ajudar a tornar o código mais legível e claro.

Além disso, this pode ser usado para chamar um construtor a partir de outro construtor na mesma classe:

11225354498 - Maria Vitória Gonçalves Vieira

```java
public class Carro {
    String cor;

    Carro() {
        this("Branco");  // Chama o segundo construtor
    }

    Carro(String cor) {
        this.cor = cor;
    }
}
```

Neste exemplo, o construtor Carro() chama o construtor Carro(String cor) usando this("Branco"). Isso significa que, se um Carro for criado sem um valor de cor especificado, ele receberá automaticamente a cor "Branco".

Herança

A herança é um dos pilares fundamentais da Programação Orientada a Objetos (OOP). Ela permite que uma classe (classe filha ou subclasse) herde os campos e métodos de outra classe (classe pai ou superclasse). Também é uma maneira eficiente de reutilizar código, e também permite adicionar mais funcionalidades a uma classe existente sem modificá-la.

A herança em Java é realizada usando a palavra-chave extends.

Por exemplo, vamos considerar uma classe Animal e uma classe Cachorro que herda de Animal.

11225354498 - Maria Vitória Gonçalves Vieira

```java
public class Animal {
    public void comer() {
        System.out.println("O animal está comendo");
    }
}

public class Cachorro extends Animal {
    public void latir() {
        System.out.println("O cachorro está latindo");
    }
}

public class Main {
    public static void main(String[] args) {
        Cachorro cachorro = new Cachorro();
        cachorro.comer(); // O animal está comendo
        cachorro.latir(); // O cachorro está latindo
    }
}
```

No exemplo acima, a classe Cachorro herda o método comer da classe Animal. Assim, um objeto Cachorro pode chamar tanto o método comer quanto o método latir.

Polimorfismo

O polimorfismo é comumente implementado em Java usando métodos de sobreposição (métodos com a mesma assinatura, mas comportamentos diferentes nas classes superclasse e subclasse) e interfaces.

Por exemplo, vamos expandir o exemplo anterior para ilustrar o polimorfismo:

11225354498 - Maria Vitória Gonçalves Vieira

```java
public class Animal {
    public void emitirSom() {
        System.out.println("O animal está emitindo um som");
    }
}

public class Cachorro extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("O cachorro está latindo");
    }
}

public class Gato extends Animal {
    @Override
    public void emitirSom() {
        System.out.println("O gato está miando");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal meuCachorro = new Cachorro();
        Animal meuGato = new Gato();

        meuCachorro.emitirSom(); // O cachorro está latindo
        meuGato.emitirSom();     // O gato está miando
    }
}
```

Neste exemplo, a classe Animal tem um método emitirSom(). As classes Cachorro e Gato, que são subclasses de Animal, sobrepõem esse método para fornecer comportamentos específicos. No método main, criamos objetos Cachorro e Gato usando referências de Animal. Quando chamamos o método emitirSom() nesses

11225354498 - Maria Vitória Gonçalves Vieira

objetos, o método apropriado para cada tipo de objeto é chamado. Isso é polimorfismo.

Interfaces

Em Java, uma interface é declarada usando a palavra-chave **interface** e contém métodos abstratos (métodos sem corpo), constantes (variáveis que são estaticamente e finalmente inicializadas), métodos padrão (métodos com corpo e declarados com a palavra-chave **default**), e métodos estáticos.

Veja um exemplo de como uma interface é definida:

```
public interface Animal {
    void comer();
    void mover();
}
```

Implementação de Interfaces:

Uma classe implementa uma interface usando a palavra-chave **implements**. Ao implementar uma interface, a classe está concordando em aderir ao contrato estabelecido pela interface, ou seja, a classe irá fornecer implementações para todos os métodos declarados na interface. Se uma classe não fornecer implementações para todos os métodos de uma interface, então essa classe deve ser declarada como abstrata.

Veja um exemplo de como implementar uma interface:

11225354498 - Maria Vitória Gonçalves Vieira

```java
public class Gato implements Animal {
    @Override
    public void comer() {
        System.out.println("O gato está comendo");
    }

    @Override
    public void mover() {
        System.out.println("O gato está se movendo");
    }
}
```

Neste exemplo, a classe Gato está implementando a interface Animal e fornecendo implementações para os métodos comer() e mover(). Uma vez que um método foi declarado em uma interface e implementado em uma classe, qualquer instância dessa classe pode ser usada como uma instância da interface. Isto é, você pode fazer Animal meuGato = new Gato();

Arrays

Um array em Java é um objeto que contém elementos do mesmo tipo. O comprimento de um array é estabelecido quando o array é criado e não pode ser alterado. As posições dos elementos de um array são acessadas por meio de índices inteiros que começam em 0 para o primeiro elemento e vão até o comprimento do array menos 1.

Declaração de Arrays:

Um array em Java é declarado da seguinte maneira:

```java
tipo[] nomeDoArray;
```

Por exemplo, você pode declarar um array de inteiros chamado numeros assim:

11225354498 - Maria Vitória Gonçalves Vieira

```
int[] numeros;
```

Inicialização de Arrays:

Depois de declarar um array, você precisa inicializá-lo. Existem várias maneiras de fazer isso.

1. Você pode inicializar um array quando o declara, especificando seus elementos:

```
int[] numeros = {1, 2, 3, 4, 5};
```

2. Você pode inicializar um array com a palavra-chave new, especificando o número de elementos que o array irá conter:

```
int[] numeros = new int[5];
```

Neste exemplo, numeros é um array que pode conter 5 inteiros. Os elementos são inicializados automaticamente com o valor padrão para o tipo do array, que é 0 para inteiros.

Manipulação de Arrays:

Uma vez que você tem um array, pode acessar e modificar seus elementos usando seus índices. Por exemplo:

```
numeros[0] = 10;
int primeiroNumero = numeros[0];
```

Neste exemplo, o primeiro elemento do array numeros é definido como 10 e, em seguida, esse valor é obtido e armazenado na variável primeiroNumero.

Java também fornece uma sintaxe especial para percorrer todos os elementos de um array:

11225354498 - Maria Vitória Gonçalves Vieira

```
for (int numero : numeros) {
    System.out.println(numero);
}
```

Neste exemplo, o loop for percorre todos os elementos do array numeros e imprime cada um deles. Este é conhecido como um loop for-each ou loop aprimorado for.

Note que, ao contrário de alguns outros idiomas, Java não verifica se os índices do array estão fora dos limites. Se você tentar acessar um índice que é menor que zero ou maior ou igual ao comprimento do array, Java lançará uma ArrayIndexOutOfBoundsException. É sua responsabilidade garantir que você não ultrapasse os limites do array.

Arrays Multidimensionais

Um array multidimensional em Java é um array cujos componentes também são arrays. Estes são também conhecidos como arrays aninhados ou arrays de arrays.

Os arrays multidimensionais podem ter duas ou mais dimensões. Um array de duas dimensões é como uma matriz com linhas e colunas. Por exemplo, você pode ter um array bidimensional para representar uma grade de células em um jogo de tabuleiro, ou para armazenar dados de uma tabela com linhas e colunas.

Declaração de Arrays Multidimensionais:

Um array multidimensional é declarado da seguinte maneira:

```
tipo[][] nomeDoArray;
```

Por exemplo, você pode declarar um array bidimensional de inteiros chamado matriz assim:

```
int[][] matriz;
```

Inicialização de Arrays Multidimensionais:

Assim como os arrays unidimensionais, você pode inicializar um array multidimensional quando o declara, ou pode usar a palavra-chave new:

```
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

Neste exemplo, matriz é um array bidimensional de inteiros com 3 linhas e 3 colunas.

```
int[][] matriz = new int[3][3];
```

Neste exemplo, matriz é um array bidimensional que pode conter 3 arrays, cada um dos quais pode conter 3 inteiros.

Manipulação de Arrays Multidimensionais:

Você pode acessar e modificar os elementos de um array multidimensional usando dois índices:

```
matriz[0][0] = 10;
int primeiroNumero = matriz[0][0];
```

Neste exemplo, o primeiro elemento do primeiro array dentro de matriz é definido como 10, e então esse valor é obtido e armazenado na variável primeiroNumero.

Java também fornece uma sintaxe para percorrer todos os elementos de um array multidimensional:

```
for (int[] linha : matriz) {
    for (int numero : linha) {
        System.out.println(numero);
    }
}
```


11225354498 - Maria Vitória Gonçalves Vieira

Neste exemplo, o loop externo for percorre todos os arrays dentro de matriz, e o loop interno for percorre todos os elementos de cada um desses arrays. Isto imprime todos os números na matriz, linha por linha.

Strings

Strings em Java são objetos que representam sequências de caracteres. A classe java.lang.String é usada para criar e manipular strings.

Imutabilidade de Strings:

Em Java, strings são imutáveis. Isso significa que, uma vez criada uma string, ela não pode ser alterada. Todas as operações que parecem modificar uma string (como concatenação ou substituição de caracteres) na verdade criam uma nova string.

Criação de Strings:

Há duas maneiras principais de criar strings em Java:

1. Atribuição direta de uma string literal a uma variável:

```
String minhaString = "Olá, Mundo!";
```

2. Usando o operador new para criar um novo objeto String:

```
String minhaString = new String("Olá, Mundo!");
```

A primeira maneira é geralmente preferida, pois é mais simples e eficiente. Java mantém um pool de strings internas para reutilizar strings literais e economizar memória.

Operações com Strings:

A classe String em Java fornece muitos métodos úteis para trabalhar com strings. Aqui estão alguns dos mais comuns:

11225354498 - Maria Vitória Gonçalves Vieira

- length(): Retorna o número de caracteres na string.
- charAt(int index): Retorna o caractere na posição especificada.
- substring(int beginIndex, int endIndex): Retorna uma nova string que é uma subparte da string original.
- concat(String str): Retorna uma nova string que é a concatenação da string original e da string fornecida.
- indexOf(String str): Retorna a posição da primeira ocorrência da string fornecida na string original, ou -1 se a string não for encontrada.
- equals(Object obj): Retorna true se a string é igual à string fornecida, false caso contrário.
- toLowerCase(): Retorna uma nova string que é a string original convertida para minúsculas.
- toUpperCase(): Retorna uma nova string que é a string original convertida para maiúsculas.
- trim(): Retorna uma nova string que é a string original com espaço em branco removido do início e do fim.

Além desses métodos, o Java também suporta o operador + para concatenar strings, e o operador == para comparar strings. No entanto, é importante lembrar que == compara a identidade dos objetos, não o seu conteúdo. Para comparar o conteúdo de duas strings, você deve usar o método equals().

Exceções

Em Java, uma exceção é um evento que interrompe o fluxo normal do programa. É uma condição anormal que ocorre durante a execução de um programa. As exceções podem ser geradas por muitos tipos diferentes de erros, como divisão por zero, acesso a um índice fora dos limites de um array, tentativa de acessar um objeto nulo, etc.

Try, Catch, Finally e Throw:

Java fornece uma estrutura para o tratamento de exceções que inclui as cláusulas try, catch, finally e a palavra-chave throw.

- Try: Um bloco try é usado para envolver o código que pode lançar uma exceção. É seguido por um ou mais blocos catch.
- Catch: Um bloco catch é usado para lidar com uma exceção. Ele segue um bloco try e contém o código que é executado quando uma exceção é lançada no bloco try.

11225354498 - Maria Vitória Gonçalves Vieira

- Finally: Um bloco finally segue um bloco try ou catch e contém o código que é sempre executado, independentemente de uma exceção ser ou não lançada.
- Throw: A palavra-chave throw é usada para lançar explicitamente uma exceção.

Exceções Verificadas e Não Verificadas:

Java classifica exceções em duas categorias: verificadas e não verificadas.

- Exceções verificadas: São exceções que o compilador pode verificar durante a compilação. São aquelas que derivam da classe Exception, mas não da RuntimeException. Elas devem ser declaradas na cláusula throws de um método ou capturadas dentro de um bloco try/catch.
- Exceções não verificadas: São exceções que o compilador não verifica durante a compilação. Estas são principalmente programáticas e representam defeitos na lógica do programa. Elas derivam da classe RuntimeException e incluem exceções como NullPointerException, ArrayIndexOutOfBoundsException, ArithmeticException, etc.

Veja um exemplo de tratamento de exceção:

```
try {
    // código que pode lançar uma exceção
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    // código que é executado quando a exceção é lançada
    System.out.println("Erro: Divisão por zero");
} finally {
    // código que é sempre executado
    System.out.println("Isso sempre será impresso");
}
```

Neste exemplo, uma ArithmeticException é lançada no bloco try porque estamos tentando dividir um número por zero. Esta exceção é capturada pelo bloco catch, que imprime uma mensagem de erro. Em seguida, o bloco finally é executado e imprime outra mensagem.

11225354498 - Maria Vitória Gonçalves Vieira

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[1].

O Garbage Collector em Java é um mecanismo automatizado que desempenha um papel fundamental na gestão de memória dentro da Java Virtual Machine (JVM). O processo de garbage collection é responsável por identificar e descartar objetos que não são mais necessários, liberando assim recursos de memória para serem reutilizados. Quando um objeto em Java não possui mais referências que apontam para ele, torna-se elegível para a coleta de lixo. O Garbage Collector opera principalmente em segundo plano, garantindo que a alocação e desalocação de memória sejam eficientemente gerenciadas sem intervenção manual, o que ajuda a evitar vazamentos de memória e outros problemas relacionados à memória.

A eficácia do Garbage Collector é complementada pela arquitetura de memória da JVM, que divide a memória em várias regiões, como Heap e Stack. Enquanto o Heap é usado para a alocação dinâmica de objetos, a Stack é utilizada para gerenciar chamadas de métodos e armazenar variáveis locais. Essa separação ajuda na organização dos dados e na otimização da performance da JVM. No contexto de aplicações de alta carga, um gerenciamento eficaz de memória é essencial para a sustentabilidade e escalabilidade, e o Garbage Collector desempenha um papel vital ao assegurar que os recursos de memória sejam geridos de forma eficiente, permitindo que a aplicação mantenha uma performance ótima mesmo sob condições de uso intenso.

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

[1] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

1. (FGV – CGU – 2022) Observe o trecho de código a seguir.

```
1 import java.io.*;
2
3 interface Segunda { public void segunda();}
4 interface Terca { public void terca();}
5 interface Quarta extends Segunda, Terca { public void quarta();}
6
7 class Quinta implements Segunda {
8 public void segunda(){System.out.println("Segunda");}
9 }
10
11 class Sexta extends Quinta implements Quarta {
12 public void segunda(){System.out.println("Segunda!");}
13 public void terca(){System.out.println("Segunda!!");}
14 public void quarta(){System.out.println("Segunda!!!");}
15 }
16
17 public class teste
18 {
19 public static void main( String[] args )
```

```
20 {
21 Quinta dia = (Quinta) new Sexta();
22 dia.segunda();
23 }
24 }
```

A saída produzida pela execução do código é:

a) Erro de compilação na linha 5

b) Segunda

c) Segunda!

d) Segunda!!

e) Segunda!!!

Comentários:

Primeiramente, temos que a assertiva a está incorreta, já que a linha 5 não gera erro, pois entre interfaces pode haver "herança múltipla", ou seja, herança de duas ou mais classes. Agora vamos analisar o código. No código há três interfaces, Segunda, Terca e Quarta além das classes Sexta e Quinta. Sendo que a classe Quinta é classe pai, ou superclasse, e a classe Sexta é classe filha, ou subclasse. O foco da questão está na chamada da linha 21 Quinta dia = (Quinta) new Sexta(); nessa linha, temos que o objeto dia chama o método construtor Sexta, que está na classe Sexta, que é filha de Quinta. Quando esse objeto for criado, ele vai à classe Sexta e procura pelo método segunda, se o encontrar ali, executa o que estiver no corpo do método, senão, ele vai à classe Quinta a procura de um método segunda. Daí, quando a linha dia.segunda(); for executada, o objeto dia procura um método segunda. Ele vai à classe Sexta, pois esse é o método construtor que foi passado na criação do objeto, após isso, caso não encontre o método segunda na classe Sexta, ele vai à classe Quinta, que deve ter um método segunda presente em seu corpo. Assim, o resultado que será apresentado na tela é Segunda! E temos nosso gabarito na alternativa C.

11225354498 - Maria Vitória Gonçalves Vieira

Gabarito: Letra C

2. (FGV – TJDFT – 2022) Observe as classes abaixo descritas na linguagem de programação Java.

```
public class DocumentoProcesso extends Object
{
   private String docNumero;
   private int classificacao;
   public DocumentoProcesso (String docNumero, int classificacao){
    if (classificacao < 1)
       throw new IllegalArgumentException (
           "A classificação do documento deve ser no mínimo 1.");
    this.docNumero = docNumero;
    this.classificacao = classificacao;
   }
   public String getDocNumero()
   { return docNumero; }
   public int getClassificacao()
   { return classificacao; }
   public void setDocNumero(String docNumero)
   { this.docNumero = docNumero; }
   public void setClassificacao(int classificacao)
   { this.classificacao = classificacao; }
```


11225354498 - Maria Vitória Gonçalves Vieira

```
    public int promove()
    { return classificacao + 1; }
}
public class Oficio extends DocumentoProcesso
{
    private int precedencia;
    public Oficio (String docNumero, int classificacao, int precedencia)
    {super(docNumero,classificacao);
    this.precedencia = precedencia; }
public int getPrecedencia()
{ return precedencia; }
public void setPrecedencia(int precedencia)
{ this.precedencia = precedencia; }
@Override
public int promove()
{ return precedencia + 1; }
}
```

Com base nos conceitos de orientação a objetos, é correto afirmar que:

a) os atributos private de DocumentoProcesso são acessíveis por Oficio;

b) a anotação @Override indica que o método promove() é abstrato e polimórfico;

c) a classe Oficio representa uma herança múltipla das classes DocumentoProcesso e Object;

11225354498 - Maria Vitória Gonçalves Vieira

d) a classe Oficio é composta pela classe DocumentoProcesso, permitindo que uma instância da classe Oficio faça referências a instâncias da classe DocumentoProcesso;

e) os métodos getDocNumero() e setDocNumero() da classe DocumentoProcesso encapsulam o atributo docNumero e asseguram que os objetos da classe mantenham estados consistentes.

Comentários:

Vamos analisar as assertivas: a letra a diz que os atributos private de DocumentoProcesso são acessíveis por Oficio. Vejamos a classe Oficio: a declaração dela é a seguinte: public class Oficio extends DocumentoProcesso. Portanto, ofício é filha da classe DocumentoProcesso. Já que os atributos possuem o modificador private, eles não podem ser compartilhados entre as classes. Lembrem-se da definição: Membros da classe definidos como private não podem ser acessados ou usados por nenhuma outra classe. Assim, temos uma assertiva errada. Letra B: a anotação @Override indica que o método promove() é abstrato e polimórfico. Errada assertiva B porque @Override indica que um método da classe-pai será sobrescrito por um método da classe-filha, ou seja, a classe filha modifica o método que foi declarado na classe pai. Para indica que o método promove() é abstrato deveria ser utilizado abstract. A letra C também está errada, pois na verdade, é a classe DocumentoProcesso que faz referência à Oficio. Por fim, temos o nosso gabarito na letra E. Pessoal, os métodos getDocNumero() e setDocNumero() de fato, são utilizados para encapsular o atributo docNumero. Ademais, asseguram que os objetos da classe mantenham estados consistentes. Getters e setters são usados para proteger seus dados, especialmente na criação de classes. Para cada instância de variável, um método getter retorna seu valor, enquanto um método setter o define ou atualiza seu valor.

Gabarito: Letra E

3. (FGV – PGE-AM – 2022) Considere o trecho de programa Java abaixo.

11225354498 - Maria Vitória Gonçalves Vieira

```
public class Calcular {
    public double reajuste(double salario, double percentual) {
        return salario + salario * percentual / 100;
    }
    public double reajuste(double salario) {
        return salario * 1.30;
    }
    public static double reajuste(float salario) {
        return salario * 0.20;
    }
}
```

É correto afirmar que

a) o código mostra um exemplo de sobrescrita de métodos, pois há vários métodos com o mesmo nome, porém, com parâmetros diferentes.

b) ocorrerá um erro no último método reajuste, pois este não pode ser estático.

c) uma chamada correta ao método reajuste, em condições ideais, é double sal = 1000.5; double c = Calcular.reajuste(sal);

d) ocorrerá um erro na classe Calcular, pois não é permitido criar vários métodos com o mesmo nome, como ocorre com o método reajuste.

e) uma chamada correta ao método reajuste, em condições ideais, é float sal = (float) 1000.5; double c = Calcular.reajuste(sal);

Comentários:

Sobrecarga ou Polimorfismo Estático ocorre quando uma classe possui métodos com mesmo nome, entretanto assinaturas diferentes. Ocorre em Tempo de Compilação. Veja que o método reajuste possui diferentes implementações. Com essa explicação inicial podemos verificar que a letra A está incorreta, pois não ocorre sobrescrita, mas sim, sobrecarga. O erro da letra B consiste em dizer que método reajuste não pode ser estático. O erro da letra C consiste na falta do "double" indicado double sal = (double) 1000.5; double c = Calcular.reajuste(sal). O erro da alternativa d é dizer que não é permitido criar vários métodos com o mesmo nome. Na verdade, não vai ocorrer um erro. Ocorre uma sobrescrita que é possível em Java! Por fim, temos nosso gabarito Letra E!

Gabarito: Letra E

11225354498 - Maria Vitória Gonçalves Vieira

4. (FGV – DPE-RJ – 2019) Considere as seguintes afirmativas sobre class constructor na linguagem Java.

I. Deve receber o mesmo nome da classe a ele associada.

II. Não deve ser especificado um tipo de retorno na sua declaração.

III. É útil para a definição de valores iniciais para os atributos da classe.

IV. É sempre declarado como public.

É correto somente o que se afirma em:

a) I e II;

b) II e III;

c) III e IV;

d) I, II e III;

e) I, III e IV.

Comentários:

Vamos lá, um construtor é um método especial usado para inicializar objetos. O construtor é chamado quando um objeto de uma classe é criado. Ele pode ser usado para definir valores iniciais para atributos de objeto. o construtor é apenas invocado no momento da criação do objeto através do operador new. A assinatura de um construtor diferencia-se das assinaturas dos outros métodos por não ter nenhum tipo de retorno (nem mesmo void). Além disto, o nome do construtor deve ser o próprio nome da classe. O construtor pode receber argumentos, como qualquer método. Usando o mecanismo de sobrecarga, mais de um construtor pode ser definido para uma classe. Assim, temos que os itens I, II e III estão corretos. Já, a assertiva IV está errada, pois pode ser public, protected ou private.

Gabarito: Letra D

5. (FGV – MPE-AL – 2018) Analise o código Java a seguir.

```
public class Exemplo {

 public static void main(String[] args) {
```

```
  int  a = 4, b = 16;
  String out = (a^b)==0 ? "Sim": (a & b)!=0
                    ? "Talvez" : "Não";
  System.out.println(out);
 }
}
```

A execução do código acima produzirá

a) um erro de compilação.

b) um erro de execução.

c) a linha “Sim”.

d) a linha “Não”.

e) a linha “Talvez”.

Comentários:

A questão está totalmente focada no operador ternário String out = (a^b)==0 ? "Sim": (a & b)!=0. Vamos destrinchá-lo. Inicialmente temos: (a^b)==0. O operador ^é um XOR que consiste em um ou exclusivo que retorna 1 se os bits forem diferentes: Portanto, temos a = 4 e b = 16. a é igual 4 que é igual a 00100 em binário e b – 16 – é igual a 10000 em binário. Fazendo a operação XOR entre os valores temos como resultado 10100, que é igual a 20 em decimal. Podemos perceber que o valor não é sim, já que 20 não é igual a 0. Portanto vamos ao próximo argumento que é (a & b)!=0., ou seja, a e b é diferente de zero. Vejamos como fica o resultado de a & b = 00000 porque o operador “E” (&) só retorna 1 se ambos bits forem 1: a - 00100 (4) b - 10000 (16) = 00000 (0). Veja que o teste vai ficar da seguinte forma: (a & b) != 0. Sabemos que a & b é 0 (de acordo com o cálculo acima. Assim fica 0 != 0 ? ou seja, zero é diferente de zero? Não né! Então é retornado o valor “Não”. Já que o operador ternário (TESTE)? X : Y retorna X, se TESTE for verdadeiro, e Y se teste for falso.

Gabarito: Letra D

11225354498 - Maria Vitória Gonçalves Vieira

QUESTIONÁRIO DE REVISÃO E APERFEIÇOAMENTO

A ideia do questionário é elevar o nível da sua compreensão no assunto e, ao mesmo tempo, proporcionar uma outra forma de revisão de pontos importantes do conteúdo, a partir de perguntas que exigem respostas subjetivas.

São questões um pouco mais desafiadoras, porque a redação de seu enunciado não ajuda na sua resolução, como ocorre nas clássicas questões objetivas.

O objetivo é que você realize uma auto explicação mental de alguns pontos do conteúdo, para consolidar melhor o que aprendeu ;)

Além disso, as questões objetivas, em regra, abordam pontos isolados de um dado assunto. Assim, ao resolver várias questões objetivas, o candidato acaba memorizando pontos isolados do conteúdo, mas muitas vezes acaba não entendendo como esses pontos se conectam.

Assim, no questionário, buscaremos trazer também situações que ajudem você a conectar melhor os diversos pontos do conteúdo, na medida do possível.

É importante frisar que não estamos adentrando em um nível de profundidade maior que o exigido na sua prova, mas apenas permitindo que você compreenda melhor o assunto de modo a facilitar a resolução de questões objetivas típicas de concursos, ok?

Nosso compromisso é proporcionar a você uma revisão de alto nível!

Vamos ao nosso questionário:

Perguntas

1. Como o conceito de "Write Once, Run Anywhere" é realizado em Java?
2. Quais são os principais tipos primitivos em Java?
3. Quais são os quatro principais modificadores de acesso em Java?
4. O que a palavra-chave 'final' faz em Java?
5. O que a palavra-chave 'this' faz em Java?
6. O que é uma interface em Java e para que serve?

7. Como você declara e inicializa um array em Java?
8. O que é exceção em Java e como você pode lidar com ela?
9. Quais são as duas principais categorias de exceções em Java?
10. Qual é o propósito do Garbage Collector em Java?
11. Como o polimorfismo é implementado em Java?
12. Qual é a diferença entre tipos primitivos e tipos de referência em Java?
13. O que a palavra-chave 'extends' faz em Java?
14. Qual é a diferença entre 'public', 'private', 'protected' e visibilidade 'default' em Java?
15. Como você pode criar múltiplos construtores para uma única classe em Java?
16. Como são declarados e usados os arrays multidimensionais em Java?
17. Como a herança é implementada em Java?
18. Quais são os principais usos das palavras-chave 'try', 'catch' e 'finally' em Java?
19. O que a palavra-chave 'new' faz em Java?
20. Quais são as principais características da linguagem Java?

Perguntas e Respostas

1. Como o conceito de "Write Once, Run Anywhere" é realizado em Java?
   Resposta: Isso é possível graças à JVM (Máquina Virtual Java) e ao bytecode. O código Java é compilado para bytecode, que pode ser interpretado e executado em qualquer máquina que tenha uma JVM.
2. Quais são os principais tipos primitivos em Java?
   Resposta: Os principais tipos primitivos em Java são int, long, short, byte, float, double, char e boolean.
3. Quais são os quatro principais modificadores de acesso em Java?
   Resposta: Os quatro principais modificadores de acesso em Java são public, private, protected e default.
4. O que a palavra-chave 'final' faz em Java?
   Resposta: A palavra-chave 'final' é usada para indicar que um valor não pode ser alterado após a inicialização, que um método não pode ser sobrescrito ou que uma classe não pode ser herdada.
5. O que a palavra-chave 'this' faz em Java?
   Resposta: 'this' é uma referência ao objeto atual em um método ou construtor.
6. O que é uma interface em Java e para que serve?
   Resposta: Uma interface em Java é um tipo de referência semelhante a uma classe, usada para agrupar métodos relacionados com corpos vazios. Serve para garantir a implementação de métodos em classes que implementam a interface.

11225354498 - Maria Vitória Gonçalves Vieira

7. Como você declara e inicializa um array em Java?
   Resposta: Você pode declarar um array em Java com o tipo de dados seguido por colchetes e o nome do array. Por exemplo: int[] array. Você pode inicializar um array usando chaves, por exemplo: int[] array = {1, 2, 3, 4, 5}.
8. O que é exceção em Java e como você pode lidar com ela?
   Resposta: Exceção é um evento que ocorre durante a execução do programa e interrompe o fluxo normal de instruções. Pode ser manipulado usando blocos try, catch e finally.
9. Quais são as duas principais categorias de exceções em Java?
   Resposta: As duas principais categorias de exceções em Java são as verificadas e não verificadas.
10. Qual é o propósito do Garbage Collector em Java?
    Resposta: O Garbage Collector em Java tem a função de gerenciar automaticamente a memória, liberando o programador da tarefa de liberar manualmente a memória alocada para objetos que não são mais usados.
11. Como o polimorfismo é implementado em Java?
    Resposta: O polimorfismo em Java é implementado através da sobrecarga de métodos (métodos com o mesmo nome, mas com diferentes listas de parâmetros) e da substituição de métodos (um método em uma classe filha tem o mesmo nome e tipo de retorno que um método na classe pai).
12. Qual é a diferença entre tipos primitivos e tipos de referência em Java?
    Resposta: Os tipos primitivos armazenam valores, enquanto os tipos de referência armazenam referências a objetos. Os tipos primitivos não têm métodos, enquanto os tipos de referência têm métodos.
13. O que a palavra-chave 'extends' faz em Java?
    Resposta: 'extends' é usada para indicar que uma classe está herdando de outra classe.
14. Qual é a diferença entre 'public', 'private', 'protected' e visibilidade 'default' em Java?
    Resposta: 'public' permite o acesso de qualquer lugar, 'private' restringe o acesso apenas à classe, 'protected' permite o acesso à classe, ao pacote e às subclasses, enquanto a visibilidade 'default' permite o acesso apenas à classe e ao pacote.
15. Como você pode criar múltiplos construtores para uma única classe em Java?
    Resposta: Você pode criar múltiplos construtores para uma única classe em Java através da sobrecarga de construtores, que é quando você define múltiplos construtores com listas de parâmetros diferentes.
16. Como são declarados e usados os arrays multidimensionais em Java?
    Resposta: Os arrays multidimensionais em Java são declarados da mesma forma que os arrays unidimensionais, mas com colchetes adicionais para cada dimensão adicional. Eles são usados como arrays de arrays.

11225354498 - Maria Vitória Gonçalves Vieira

17. Como a herança é implementada em Java?
    Resposta: A herança é implementada em Java usando a palavra-chave 'extends'. Uma classe filha que herda de uma classe pai pode acessar todos os campos e métodos da classe pai que não são privados.
18. Quais são os principais usos das palavras-chave 'try', 'catch' e 'finally' em Java?
    Resposta: 'try' é usada para cercar o código que pode lançar uma exceção, 'catch' é usado para lidar com a exceção e 'finally' é usado para executar código independentemente de uma exceção ter sido lançada ou não.
19. O que a palavra-chave 'new' faz em Java?
    Resposta: 'new' é usada para criar uma nova instância de uma classe ou um novo array em Java.
20. Quais são as principais características da linguagem Java?
    Resposta: As principais características de Java incluem ser orientada a objetos, portável, segura, robusta e de alto desempenho.

Lista de Questões Estratégicas

1. (FGV – CGU – 2022) Observe o trecho de código a seguir.

```
1 import java.io.*;

2

3 interface Segunda { public void segunda();}

4 interface Terca { public void terca();}

5 interface Quarta extends Segunda, Terca { public void quarta();}

6

7 class Quinta implements Segunda {

8 public void segunda(){System.out.println("Segunda");}

9 }

10
```


11225354498 - Maria Vitória Gonçalves Vieira

```
11 class Sexta extends Quinta implements Quarta {

12 public void segunda(){System.out.println("Segunda!");}

13 public void terca(){System.out.println("Segunda!!");}

14 public void quarta(){System.out.println("Segunda!!!");}

15 }

16

17 public class teste

18 {

19 public static void main( String[] args )

20 {

21 Quinta dia = (Quinta) new Sexta();

22 dia.segunda();

23 }

24 }
```

A saída produzida pela execução do código é:

a) Erro de compilação na linha 5

b) Segunda

c) Segunda!

d) Segunda!!

e) Segunda!!!

2. (FGV – TJDFT – 2022) Observe as classes abaixo descritas na linguagem de programação Java.

```
public class DocumentoProcesso extends Object
{
    private String docNumero;
    private int classificacao;
    public DocumentoProcesso (String docNumero, int classificacao){
      if (classificacao < 1)
         throw new IllegalArgumentException (
             "A classificação do documento deve ser no mínimo 1.");
       this.docNumero = docNumero;
       this.classificacao = classificacao;
      }
      public String getDocNumero()
      { return docNumero; }
      public int getClassificacao()
      { return classificacao; }
      public void setDocNumero(String docNumero)
      { this.docNumero = docNumero; }
      public void setClassificacao(int classificacao)
      { this.classificacao = classificacao; }
      public int promove()
      { return classificacao + 1; }
}
```


11225354498 - Maria Vitória Gonçalves Vieira

```
public class Oficio extends DocumentoProcesso
{
    private int precedencia;
    public Oficio (String docNumero, int classificacao, int precedencia)
    {super(docNumero,classificacao);
    this.precedencia = precedencia; }
public int getPrecedencia()
{ return precedencia; }
public void setPrecedencia(int precedencia)
{ this.precedencia = precedencia; }
@Override
public int promove()
{ return precedencia + 1; }
}
```

Com base nos conceitos de orientação a objetos, é correto afirmar que:

a) os atributos private de DocumentoProcesso são acessíveis por Oficio;

b) a anotação @Override indica que o método promove() é abstrato e polimórfico;

c) a classe Oficio representa uma herança múltipla das classes DocumentoProcesso e Object;

d) a classe Oficio é composta pela classe DocumentoProcesso, permitindo que uma instância da classe Oficio faça referências a instâncias da classe DocumentoProcesso;

e) os métodos getDocNumero() e setDocNumero() da classe DocumentoProcesso encapsulam o atributo docNumero e asseguram que os objetos da classe mantenham estados consistentes.

11225354498 - Maria Vitória Gonçalves Vieira

3. (FGV – PGE-AM – 2022) Considere o trecho de programa Java abaixo.

```
public class Calcular {
    public double reajuste(double salario, double percentual) {
        return salario + salario * percentual / 100;
    }
    public double reajuste(double salario) {
        return salario * 1.30;
    }
    public static double reajuste(float salario) {
        return salario * 0.20;
    }
}
```

É correto afirmar que

a) o código mostra um exemplo de sobrescrita de métodos, pois há vários métodos com o mesmo nome, porém, com parâmetros diferentes.

b) ocorrerá um erro no último método reajuste, pois este não pode ser estático.

c) uma chamada correta ao método reajuste, em condições ideais, é double sal = 1000.5; double c = Calcular.reajuste(sal);

d) ocorrerá um erro na classe Calcular, pois não é permitido criar vários métodos com o mesmo nome, como ocorre com o método reajuste.

e) uma chamada correta ao método reajuste, em condições ideais, é float sal = (float) 1000.5; double c = Calcular.reajuste(sal);

4. (FGV – DPE-RJ – 2019) Considere as seguintes afirmativas sobre class constructor na linguagem Java.

I. Deve receber o mesmo nome da classe a ele associada.

II. Não deve ser especificado um tipo de retorno na sua declaração.

III. É útil para a definição de valores iniciais para os atributos da classe.

IV. É sempre declarado como public.

É correto somente o que se afirma em:

11225354498 - Maria Vitória Gonçalves Vieira

a) I e II;

b) II e III;

c) III e IV;

d) I, II e III;

e) I, III e IV.

5. (FGV – MPE-AL – 2018) Analise o código Java a seguir.

```
public class Exemplo {
 public static void main(String[] args) {
  int  a = 4, b = 16;
  String out = (a^b)==0 ? "Sim": (a & b)!=0
                    ? "Talvez" : "Não";
  System.out.println(out);
 }
}
```

A execução do código acima produzirá

a) um erro de compilação.

b) um erro de execução.

c) a linha “Sim”.

d) a linha “Não”.

e) a linha “Talvez”.

Gabaritos

1. C
2. E
3. E
4. D
5. D

11225354498 - Maria Vitória Gonçalves Vieira

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. 

Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. 

Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. 

Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. 

Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. 

Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. 

Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. 

Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. 

O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.